

1 · FUNÇÕES (envolvem o valor)	
<code>int("123") → 123</code>	texto → inteiro; mata zero à esq. (<code>int("02") → 2</code>)
<code>float("12.5") → 12.5</code>	texto → decimal (ponto, NUNCA vírgula)
<code>str(42) → "42"</code> <code>input("msg")</code> <code>print(x, y)</code>	número → texto lê do usuário; retorna SEMPRE str mostra; vírgula põe espaço entre args
<code>len(x)</code>	tamanho: lista, string, df (nº de linhas)
<code>round(x, 2)</code> <code>sum(lista) / max / min</code> <code>range(5)</code>	arredonda em 2 casas soma / maior / menor de uma lista 0,1,2,3,4 (pra <code>for i in range(n)</code>)
<code>type(x)</code>	debug: mostra o tipo da variável
⚠ DATAFRAME RETORNA , não altera: sempre <code>x = int(x)</code> , nunca <code>int(x)</code> solto.	
2 · MÉTODOS DE STRING (depois)	
<code>s.split(" ")</code>	corta → LISTA de str (converter números!)
<code>s.upper() / s.lower()</code> <code>s.strip()</code> <code>s.isdigit()</code>	MAIÚSCULA / minúscula (atribuir!) tira espaços das pontas True só se TUDO é dígito ("12.3", "-5", "12" → False)
<code>s.replace("a", "b")</code> <code>s[0:4]</code>	troca todas as ocorrências fatia: posições 0,1,2,3 (fim NÃO entra)
<code>s[6:8]+"/"+s[4:6]</code> <code>"x" in s / in lista</code>	remontar data fatiada contém?
<code>a, b, c = s.split("/")</code>	desempacota: 1 nome por pedaço (nº igual!)
3 · F-STRING (montar texto)	
<code>f"01 {nome}"</code> <code>f"x{:.2f}"</code> <code>f"x{:.1f}"</code>	<code>f</code> liga o modo; <code>{}</code> injeta variável 2 casas decimais (mata 3.59999...) 1 casa decimal
⚠ UMA string contínua . Sem <code>+</code> , sem <code>{}</code> fora do <code>f"</code> . Escreve a frase literal primeiro, depois põe as chaves nas variáveis. Caractere especial do enunciado (-): COPIA.	
4 · CRIAR / LER DATAFRAME	
<code>import pandas as pd</code> <code>df = pd.read_excel("a.xlsx")</code> <code>pd.read_excel("a.xlsx", index_col=0)</code> <code>pd.read_csv("a.csv")</code>	sempre primeiro planilha Excel 1ª coluna vira índice CSV padrão (vírgula separa, ponto decimal)
<code>pd.read_csv("a.csv", sep=";", decimal=",")</code>	CSV BRASILEIRO : sem isso nº vira texto
<code>df = pd.DataFrame({ "time": ["Palmeiras", "Flamengo"], "pontos": [68, 68]})</code> # do zero: dict # chave=coluna, lista=valores	
⚠ Teste pós-leitura : <code>df.dtypes</code> . Coluna de valor tem que ser <code>float64/int64</code> ; se <code>object</code> = virou texto, faltou <code>decimal=","</code> .	
5 · EXPLORAR	
<code>df.head() / df.tail()</code>	5 primeiras / 5 últimas linhas (DF INTEIRO, todas col.)
<code>df.shape</code> <code>df.columns / list(df.columns)</code>	(linhas, colunas), sem parênteses nomes EXATOS das colunas (acento conta!)
<code>df.dtypes</code> <code>df.info() / df.describe()</code>	tipo de cada coluna resumo geral / stats completos

6 · SELECIONAR / FILTRAR / ALTERAR	
<code>df["col"]</code> <code>df["col"] == "SP"</code> <code>df[df["col"] == "SP"]</code> <code>df.loc[mask, "col"] = v</code>	uma coluna (Series) MÁSCARA: True/False por linha (!= >= <...) só as linhas True (tabela filtrada) REGRA DE OURO p/ trocar valor filtrado
<code>df.loc[mask, "col"] += 0.04</code> <code>df["nova"] = df["a"] * df["b"]</code> <code>linha = df[df["id"]==x].iloc[0]</code>	somar só nos filtrados criar coluna (vetorizado, sem laço) UMA linha como ficha → <code>linha["col"]</code>
<code>x in df["col"].values</code> <code>df["col"].isin(["A", "B"])</code> <code>df["col"].astype(str)</code>	existe? (SEM .values checa o índice!) máscara: está na lista? converte coluna (p/ comparar nº com texto)
<code>df["col"].str[0:3]</code> <code>df["col"].str.upper()</code>	fatia cada linha (versão coluna do <code>s[0:3]</code>) str dá os métodos de string na coluna
⚠ NUNCA <code>df[df[...]==x]["col"] = v</code> (cópia, não altera). FutureWarning/Chained na prova: IGNORAR (enunciado manda).	
7 · AGREGAR / CONTAR	
<code>df["col"].sum()</code> <code>df["col"].agg(["min", "mean", "max"])</code>	soma, média, mín, máx vários stats em UMA operação
<code>df["col"].value_counts()</code>	contagem POR categoria (tabela pronta)
<code>df["col"].nunique() / .unique()</code>	QUANTAS distintas / QUAIS são
<code>mask.sum()</code>	conta True (True=1); .count() = não-nulos, NÃO use
<code>serie.sort_values()</code>	crescente; <code>ascending=False</code> = decrescente
<code>serie.head(5)</code>	top 5 do ranking
8 · ANÁLISE (Excel → pandas)	
<code>df.groupby("g")["v"].mean()</code> <code>df.groupby("g")["v"].agg(["mean", "max"])</code> <code>pd.merge(a, b, on=["k1", "k2"])</code>	média de v POR grupo (+ .sort_values(ascending=False) = ranking) vários stats por grupo PROCV: traz colunas de b casando pelas chaves. Chave faltando MULTIPLICA linhas (teste: <code>shape</code>)
<code>df.pivot_table(index="Estado", columns="Produto", values="Total", aggfunc="mean").reset_index()</code>	Tabela Dinâmica: <code>index</code> =Linhas, <code>columns</code> =Colunas, <code>values</code> =Valores, <code>aggfunc</code> =Resumir por . <code>reset_index()</code> devolve coluna comum; <code>df.columns.name=None</code> limpa rótulo
<code>(df["m"] < 0).replace({True:"prejuízo", False:"lucro"})</code> <code>np.where(cond, "sim", "não")</code>	classificar sem if idem (exige <code>import numpy as np</code>)
⚠ TRIAGEM : dataset grande + média/ranking/cruzar = pandas SEM for. Lista de strings item a item = for + split. Escreveu <code>for</code> em análise? Pergunta: "qual ferramenta pandas faz isso?"	

9 · ESQUELETOS
<pre># VALIDAÇÃO (valida ANTES de converter!) d = input("Dist: ") uf = input("UF: ").upper() if not d.isdigit() or uf not in ["D","N"]: print("ERRO DE ENTRADA") else: d = int(d) if d <= 2000: # "até X" INCLUI X taxa = 5.0 elif d <= 8000: # elif = faixa seguinte taxa = d/1000 * 1.2 # 1.2 PONTO! else: taxa = 25.0 if uf == "N": taxa *= 2 # dobro vale p/ fixo tb print(f"R\$ {taxa:.2f}")</pre>
<pre># ACUMULADOR (Q5: lista de strings) cont = 0; total = 0 # zera FORA for item in lista: # item = 1 string partes = item.split(" ") # da FRUTA, não da cesta valor = float(partes[2]) # converte E atribui cont += 1 total += valor print(f"... {partes[0]} ...") # 1 por item print(f'{cont} x - Total: R\$ {total:.2f}') # FORA (desindenta =</pre>
<pre># DATA C/ CAMPO OPCIONAL (split em estágios) meses = ['janeiro','fevereiro','março','abril', 'maio','junho','julho','agosto','setembro', 'outubro','novembro','dezembro'] for reg in datas: partes = reg.split(" ") # grosso 1º dia, mes, ano = partes[0].split("/") txt = f"{int(dia)} de {meses[int(mes)-1]} de {ano}" if len(partes) == 2: # guarda! h, m = partes[1].split(":") h = int(h); m = int(m) txt += " à 1 hora" if h==1 else f" às {h} horas" if m == 1: txt += " e 1 minuto" elif m > 1: txt += f" e {m} minutos" print(txt) # m==0: omite</pre>
<pre># BUSCA POR ID + ÁRVORE (ordem = prioridade) pid = int(input("id: ")) # int: coluna é nº! if pid not in df["id"].values: print("Paciente não encontrado") else: linha = df[df["id"]==pid].iloc[0] imc = linha["peso"]/linha["altura"]**2 if linha["sist"] >= 140 or linha["diast"] >= 90: r = "consulta imediata" # regra 1 GANHA elif imc >= 30 and linha["fum"]=="sim": r = "risco alto" elif imc >= 30 or linha["idade"] >= 60: r = "risco moderado" # OU do enunciado = or else: r = "risco baixo" print(f"Paciente {pid} (IMC: {imc:.1f}) - Risco: {r}")</pre>

O ERRO-MESTRE (5x na mesma noite)
Conversão sem atribuir / input é str. <code>int(x)</code> e <code>x.upper()</code> soltos NÃO mudam nada; <code>"1" == 1</code> é False; <code>"12"~1</code> e <code>"5"<18</code> são TypeError; <code>meses[mes-1]</code> com <code>mes</code> str quebra. REGRA: depois de TODO input ou split, pergunta "converto pra quê?" e faz <code>x = int(x)</code> na MESMA linha. Função retorna, variável só muda com <code>=</code>.
CONDIÇÕES (4 erros)
<code>is not</code> não compara valor. String se compara com <code>==</code> / <code>!=</code>. O warning do Python te DIZ o conserto: lê. <code>x != "D"</code> or <code>"N"</code> é sempre True. <code>"N"</code> sozinho é truthy. Cada lado do or/and precisa ser comparação COMPLETA. Idioma: <code>x not in ["D","N"]</code>. Com <code>!=</code> o conector é <code>and</code>. Conectivo do enunciado traduzido errado. <code>"OU" = or</code>, <code>"E" = and</code>, literal. E a ORDEM das regras é a prioridade do elif: regra 1 ganha da 2. Borda: "até X" INCLUI X. É <code><=</code>, não <code><</code>. Com <code><</code>, a entrada exata (2000, 8000, 15000) cai na faixa errada. Rubrica sem parcial: 3 bordas erradas = 1,5pt = zero.
NÚMEROS (2)
<code>1,2</code> é TUPLA, não um-vírgula-dois. Decimal é PONTO: <code>1.2</code>. Se imprimir <code>(3.0, 2)</code>, é vírgula perdida em conta. <code>3.5999999999999996</code> NÃO é bug teu. Float binário não representa 1.2 exato. Não "conserta" a conta: formata com <code>{x:.2f}</code> e segue.
TABELA x LINHA x ITEM (3x o mesmo crash)
ValueError: truth value of a Series is ambiguous = você pôs a COLUNA inteira num if. · Depois de <code>linha = df[...].iloc[0]</code>: TUDO do paciente vem de <code>linha["col"]</code>, nunca <code>df["col"]</code>. · Dentro de <code>for i in coluna</code>: o valor da vez é <code>i</code>, compara com <code>i</code>. <code>df = todo mundo · linha = um registro · i = o item da vez</code>
<code>in sem .values</code> checa o ÍNDICE. Existência de valor: <code>x in df["col"].values</code>, com tipo batendo (int com int). <code>.count()</code> em máscara conta não-nulos (tudo), não os True. Contar True = <code>mask.sum()</code>.
LAÇOS E STRINGS (5)
<code>Split na CESTA: lista.split()</code> não existe. O for entrega 1 item por volta; split é no item. Dois separadores num split só não rola (<code>" / " " "</code> vira <code> / </code>). Split em ESTÁGIOS: espaço primeiro (separa mundos), barra depois (abre a data). Campo opcional fora da guarda = IndexError. <code>partes[1]</code> só existe se <code>len(partes)==2</code>; acessa SÓ dentro do if. (Gêmeo do "Paciente não encontrado"). <code>return</code> não fecha laço. Bloco fecha DESIDENTANDO (volta à margem). <code>return</code> é só de função (def). Misturar + <code>com {}</code> fora de f-string. Escolhe UM método: f-string inteira. Justaposição (<code>"a" x</code>) não existe; literais adjacentes (<code>"a" "b"</code>) colam em silêncio.

PANDAS / MÉTODO (4)

Laço manual com contadores onde era `value_counts()`. Reflexo Excel de fazer na mão. Análise = 1 linha vetorizada (triagem da frente, seção 8).

Método inventado (`.pos()`) e fatia em parênteses. Fatia (`0:3`) SÓ em colchetes. Não lembra o método? Olha a seção 6/7 da frente, não inventa.

Literal errado: "Potes" ≠ "Pote". String é EXATA. Os valores reais saem de `df["col"].unique()` ou do `head()`.

Trabalho duplicado nos 2 ramos do `if/else`. O que é comum vem ANTES do `if`; o `if` só guarda a divergência.

LEITURA DE QUESTÃO (3)

`head()` mostra o DF INTEIRO (5 linhas, TODAS as colunas), não "a coluna". Precisão custa 0,33/bloco.

Multiplicador: `*0.85 = REDUZ 15%` (preço vira 85%). `*0.2 =` vira 20% = reduz 80%. Lê o número antes de escrever a redução.

Responder Q2 em frase corrida única. Rubrica é por bloco: responde em 3 linhas, 1 por bloco (carrega / opera / mostra).

⚡ PROTOCOLO DE PROVA

0. Q0 primeiro

1. Triagem 60s

2. Ordem

3. Antes do código

4. Consertos em lote

5. Teste de fechamento

6. Warnings

7. Variáveis dadas

8. Formato exato

9. Antes de entregar

nome, turma AE_, código; não apagar os #

classifica as 5 questões: conceito / ler pandas / if+validação / DataFrame / parse em laço / análise Q1→Q2 (rápidas) → Q5 (3pts) → Q3 → Q4. Travou 10min: pula e volta

`df.columns + df.head()`: nome EXATO das colunas e valores reais

achou 3 problemas? conserta os 3, DEPOIS roda. Não 1 por vez

exemplo do enunciado + borda exata de cada faixa + caso de ERRO + os 2 períodos/categorias. SÓ aí "tá pronto"

SyntaxWarning te DIZ o conserto; FutureWarning de pandas: ignora (enunciado manda)

usa `reajuste/mes` do enunciado, NUNCA `hardcode` do exemplo ("deve funcionar pra qualquer cenário")

copia travessão/símbolo do enunciado; `{x:.2f}` quando pedir casas; confere espaço por espaço

Restart & Run All: roda tudo de cima a baixo sem erro?